



Australian
National
University

COMP8620

Advanced Topics in AI

Assignment 2 S2 2025

Author	Yang LIN
--------	----------

UID	U7418662
-----	----------

Date	17 October 2025
------	-----------------

PART A

- 1) If the discount factor γ is set to be larger, the lower bound on the number of iterations to converge to ε -close optimal solution will be **larger**.

This is because to ensure that we are ε close to optimal, the number of iterations must satisfy:

$$t \geq \left\lceil \frac{\log\left(\frac{2R_{max}}{\varepsilon(1-\gamma)}\right)}{\log\left(\frac{1}{\gamma}\right)} \right\rceil$$

where R_{max} is the largest possible reward in the MDP; $0 < \gamma < 1$; ε is small.

As γ gets closer to 1, the **denominator** $\log\left(\frac{1}{\gamma}\right)$ decreases, while the **numerator** $\log\left(\frac{2R_{max}}{\varepsilon(1-\gamma)}\right)$ increases (since $(1 - \gamma)$ becomes small, ε is small), so the overall fraction becomes larger, meaning more iterations are required.

Therefore, the closer γ is to 1 and the smaller ε is, the **more iterations** t are required for value iteration to be guaranteed ε -close to optimal.

- 2) a) Before running the Algorithm 1, the following pre-processing is required:

1. **Discretise the belief space:**

We first discretize the continuous belief space.

Each belief b is a probability distribution over all possible states in states S_p .

Since there are infinitely many possible beliefs, we generate a **finite representative set**

$B = [b_1, b_2, \dots, b_m]$, $m \in \mathbb{R}^+$, using uniform gridding.

Each b_m is normalised so that $\sum_{s \in S_p} b_m(s) = 1$, $b_m(s) \geq 0$ and stored as a discrete state.

2. **Compute expected rewards for each (b, a) pair:**

For every **discretised belief state** $b_m \in B$ and every **action** $a \in A_p$, precompute the **expected immediate reward**: $R_B(b_m, a) = \sum_{s \in S_p} b_m(s) R_p(s, a)$.

- $b_m(s)$: probability (under belief b_m) that the true state is s .

- $R_p(s, a)$: POMDP's immediate reward R_p for taking a in state s .

This is a probability-weighted average of POMDP rewards, and the result $R_B(b_m, a)$ is stored.

3. **Pre-compute belief transitions**

For each discretised belief state $b_m \in B$, each action $a \in A_p$, and each possible observation $o \in O_p$, we compute each updated belief $b'(s')$ using the **Bayesian belief update formula**:

$$\tau(b, a, o)(s') = b'(s') = \frac{Z_p(o|s', a) \sum_{s \in S_p} T_p(s'|s, a) b(s)}{\sum_{s'' \in S_p} Z_p(o|s'', a) \sum_{s \in S_p} T_p(s''|s, a) b(s)}$$

Next, we map the continuous updated belief b' to the **nearest discrete belief** B using:

$$\tau(b, a, o) = \text{the nearest discretized belief to the updated } b'.$$

In this case, the probability of transitioning to b' given (b, a, o) is defined as:

$$P(b' | b, a, o) = \begin{cases} 1, & \text{if } \tau(b, a, o) = b' \\ 0, & \text{otherwise} \end{cases}$$

We pre-compute each observation probability:

$$\mathbb{P}(o|b, a) = \sum_{s'} Z_p(o|s', a) \sum_s T_p(s'|s, a) b(s)$$

Therefore, the overall transition probability in the **belief MDP** is obtained by summing over all possible observations that lead to b' : $T_B(b, a, b') = \sum_{o \in O_p, s.t. \tau(b, a, o) = b'} \mathbb{P}(o|b, a)$,

and the resulting $T_B(b, a, b')$ values are stored in a lookup table.

The input to algorithm 1 (MDP form):

- **States S:** a finite, discretized set of belief states: $B = \{b_1, \dots, b_m\}; m \in \mathbb{R}^+$.
- **Actions A:** the same action space as the POMDP, $A = A_p$.
- **Reward R:** The expected immediate reward under a belief b and action a : $R_B(b_m, a)$.
- **Transition T:** The transition function $T_B(b_m, a, b')$ represents the probability of transitioning from one belief point b to another b' after taking action a .

b)

Case1: Fully pre-processed belief-MDP (direct input to Algorithm 1)

If all preprocessing from part (a) is completed before running the algorithm, that is the belief space $B = \{b_1, \dots, b_m\}$ is discretised and the transition and reward functions T_B and R_B are precomputed, then Algorithm 1 runs exactly like a standard MDP value iteration:

for all $b_m \in B$ do

$$V^{t+1}(b_m) = \max_{a \in A_p} \left(R_B(b_m, a) + \gamma \sum_{b' \in B} T_B(b_m, a, b') V^t(b') \right)$$

Since the algorithm iterates over all belief states and all actions:

- There are $|B|$ belief states.
- For each belief, we test all actions $|A_p|$.
- For each (b_m, a) , we sum over all next beliefs $b' \in B, |B|$.

Hence the time complexity per iteration is: $O(|B|^2 \cdot |A_p|)$.

However, preprocessing becomes computationally expensive in this case.

Case2: On-the-fly observation backup ($P(o | b_m, a)$)

If we compute the observation backup dynamically inside the Bellman update, the value update

for all $b_m \in B$ do

$$V^{t+1}(b_m) = \max_{a \in A_p} \left(R_B(b_m, a) + \gamma \sum_{b' \in B} \left(\sum_{o \in O_p, s.t. \tau(b, a, o) = b'} \mathbb{P}(o|b, a) \right) V^t(b') \right)$$

For **each belief state** $b \in B$:

1. To iterate over **all actions** $a \in A_p$, ($|A_p|$).
2. For each action, to compute the belief transition term $T_B(b, a, b')$, summing over **all possible observations** $o \in O_p$ that lead to a new belief b' , ($|O_p|$).
3. Each observation probability $P(o | b, a)$ itself expands to nested sums over the underlying **POMDP state space** S_p :

$$P(o | b, a) = \sum_{s' \in S_p} Z_p(o | s', a) \sum_{s \in S_p} T_p(s' | s, a) b(s)$$

which results in a cost of $O(|S_p|^2)$ per observation.

4. Additionally, since the Bellman update includes a summation over all possible next beliefs $b' \in B$, the overall **inner loop complexity per belief** is:

$$O(|B| \cdot |A_p| \cdot |O_p| \cdot |S_p|^2)$$

Since the algorithm iterates over all $|B|$ beliefs, the total cost per iteration is:

$$O(|B|^2 \cdot |A_p| \cdot |O_p| \cdot |S_p|^2)$$

- 3) Given an infinite horizon MDP framing, the optimal policy is stationary:

$$E\left[\sum_{t=0}^{\infty} \gamma^t R_t(s, a) \mid \pi, s_0\right]; \quad 0 < \gamma < 1$$

Case1: staying still forever:

- Reward = 0 at every step;
- Total Discounted return = 0.

Case2: moving to the goal:

- Each step costs -1 ;
- Suppose it takes n steps ($8 \leq n \leq 15$).
- At step n , the robot receives reward R_g , discounted by γ^n .

Return:

$$\begin{aligned} G &= \sum_{t=0}^{\infty} \gamma^t R_t(s, a) \\ &= \sum_{t=0}^{n-1} \gamma^t (-1) + \gamma^n R_g \\ &= -\frac{1-\gamma^n}{1-\gamma} + \gamma^n R_g \end{aligned}$$

(since $\sum_{t=0}^{n-1} \gamma^t = \frac{1-\gamma^n}{1-\gamma}$ via geometric series)

We want reaching the goal to be better than staying:

$$\begin{aligned} G &= -\frac{1-\gamma^n}{1-\gamma} + \gamma^n R_g \geq 0 \quad \forall n \in [8, 15] \\ \gamma^n R_g &\geq \frac{1-\gamma^n}{1-\gamma} \\ R_g &\geq \frac{1-\gamma^n}{(1-\gamma)\gamma^n} \end{aligned}$$

Let $\gamma = 0.95, n = 8$:

$$R_g \geq \frac{1-0.95^8}{(1-0.95)0.95^8} \approx 10.15$$

Let $\gamma = 0.95, n = 15$:

$$R_g \geq \frac{1-0.95^{15}}{(1-0.95)0.95^{15}} \approx 23.17$$

To guarantee that the robot prefers reaching the goal (even in the worst case of 15 steps):

$$R_g \geq 23.17$$

So the reward for reaching the goal should be at least about 24.

4) **POMDP model:**

- **State space: never be known exactly;**
 $S = \{\text{high, medium, low}\};$
- **Action space:**
 $A = \{\text{GentleReminder, DiseaseWarning, SternYelling}\};$
- **Observation space:**
 $O = \{\text{HighAct, ModAct, LowAct}\};$
- **Transition function:**
The user's **motivation level is assumed static** within this version of the app (no mention of motivation changing over time).
Therefore: $P(s' | s, a) = \begin{cases} 1 & \text{if } s' = s \\ 0 & \text{otherwise} \end{cases}$
- **Observation function: after the action, the observation is not deterministic.**
Given the true state (motivation level), the observed activity comes from the sensor:
If state = high $\rightarrow$ user really is highly active:
 $Z(\text{HighAct} | \text{high}, a) = 0.6$

$$Z(\text{ModAct} | \text{high}, a) = 0.2$$

$$Z(\text{LowAct} | \text{high}, a) = 0.2$$

If state = medium → user really is Moderately active:

$$Z(\text{HighAct} | \text{medium}, a) = 0.2$$

$$Z(\text{ModAct} | \text{medium}, a) = 0.6$$

$$Z(\text{LowAct} | \text{medium}, a) = 0.2$$

If state = low → user really is almost inactive:

$$Z(\text{HighAct} | \text{low}, a) = 0.2$$

$$Z(\text{ModAct} | \text{low}, a) = 0.2$$

$$Z(\text{LowAct} | \text{low}, a) = 0.6$$

- **Reward function: $R(s,a)$**

$R(\text{high}, \text{GentleReminder}) = 10$ (Gentle reminder for highly motivated user)

$R(\text{medium}, \text{DiseaseWarning}) = 10$ (Warning for medium-motivated user)

$R(\text{low}, \text{SternYelling}) = 10$ (Stern warning for low-motivated user)

$R(\text{high}, \text{DiseaseWarning}) = -5$ (unnecessary warning)

$R(\text{high}, \text{SternYelling}) = -10$ (overly harsh)

$R(\text{medium}, \text{GentleReminder}) = -5$ (not effective enough)

$R(\text{medium}, \text{SternYelling}) = -5$ (unnecessarily harsh)

$R(\text{low}, \text{GentleReminder}) = -10$ (ineffective)

$R(\text{low}, \text{DiseaseWarning}) = -5$ (not effective enough)

- **Initial belief:**

Uniform if nothing is known:

$$b_0(\text{high}) = b_0(\text{medium}) = b_0(\text{low}) = \frac{1}{3}.$$

PART B

1) MDP framing:

Problem: centralised, fully observable, stochastic navigation for k drones on a 3D grid with obstacles and inter-drone collisions.

- **State S:**

$$S = \{(P_1, P_2, \dots, P_k) | P_k = (x_k, y_k, z_k)\}$$

in $\mathbb{R}^3$:

$$P_k = (x_k, y_k, z_k) | x_k \in \{0, 1, \dots, n_x - 1\}, y_k \in \{0, 1, \dots, n_y - 1\}, z_k \in \{0, 1, \dots, n_z - 1\}$$

in $\mathbb{R}^{3k}$:

the overall state space is the Cartesian product of all position components:

$$S = P_1 \times P_2 \times \dots \times P_k$$

Hence, each state $s \in S$ represents the **joint state** of k drones:

$$s = \{(x_1, y_1, z_1), (x_2, y_2, z_2), \dots, (x_K, y_K, z_K)\}$$

- **Action A:**

$$A = \{(a_1, a_2, \dots, a_k) | a_k = (\Delta x, \Delta y, \Delta z)\}$$

in $\mathbb{R}^3$:

There are 26 neighbours of the cell, excluding 'stay': $3 \times 3 \times 3 - 1 = 26$.

$$a_k = \{(\Delta x, \Delta y, \Delta z) | \Delta x, \Delta y, \Delta z \in \{-1, 0, 1\}, (\Delta x, \Delta y, \Delta z) \neq (0, 0, 0)\}$$

in $\mathbb{R}^{3k}$:

the overall **joint action space** is the Cartesian product of all individual action spaces:

$$A = a_1 \times a_2 \times \dots \times a_K = \prod_{k=1}^K a_k ; |A| = 26^k$$

Hence, each state $a \in A$ represents the **joint action** of k drones:

$$a = (a_1, a_2, \dots, a_K)$$

- **Transition T:**

Given the known stochastic transition model $T_{known}(s, a, s')$:

Let

- $s = (p_1, \dots, p_K)$ and $s' = (p'_1, \dots, p'_K)$ be joint states, where $p_k = (x_k, y_k, z_k)$.
- $a = (a_1, \dots, a_K)$ be the joint action (each a_k is one of the 26 moves for drone k).
- $T_{known}^{(i)}(p_i, a_i, p'_i)$ be the given single-agent transition kernel.
- s_{crash} represents any state where a drone moves into an obstacle or shares a cell with another drone.
- s_{goal} represents the state where all drones reach their respective goal cells.

Then, the **joint transition probability** is defined as:

$$T(s, a, s') = \begin{cases} 1 & \text{if } s \in \{s_{goal}, s_{crash}\} \text{ and } s' = s \text{ (absorbing state)} \\ \prod_i^k T_{known}(p_i, a_i, p'_i) & \text{if } s' \text{ is valid (no collision, within bounds, not obstacle)} \\ 0 & \text{otherwise} \end{cases}$$

where

-The **absorbing condition** ensures that once all agents reach their goals or a collision occurs, the process terminates and remains in that state.

- The **joint probability** is computed as the product of all agents' individual transitions, assuming independence given their actions.

- **Reward R:** $\forall a \in A, s \in S: R = R_{terminal} + R_{step}$

-> **Terminal rewards** $R_{terminal}$:

Success: $R_{terminal}(s, a, s_{goal}) = +100$ when all drones reach their goal cells.

Collision: $R_{terminal}(s, a, s_{collision}) = -50$ if any drone collides with an obstacle or another drone.

-> **Step cost** R_{step} :

Small step penalty: $R_{step}(s, a, s') = -1$ for each movement to encourage efficiency.

2) **Algorithm name:** Monte Carlo Tree Search with UCT and progressive widening

Detailed specification of MCTS four components:

1. **Selection:** `\_uct\_select()`

Method: UCT (upper confidence bound for trees)

Formula: Choose an action a^* to perform at state s as

$$a^* = \pi_{UCT}(s) = \arg \max_{a \in A} Q(s, a) + c \sqrt{\frac{\ln(n(s))}{n(s, a)}}$$

where c : a constant indicating how to balance exploration and exploitation

$Q(s, a)$: the average total discounted reward obtains by taking action a from state s

$n(s)$: # times node s has been visited;

$n(s, a)$: # times the out-edge of s with label a has been visited

2. **Expansion:**

Purpose: Decide **when** and **how** to add new child nodes (try new actions) to the MCTS tree.

Method: Progressive widening with intelligent action sampling

- **Progressive widening** formula: (controls “when”)

$$\max_children(s) = \text{floor}(k \cdot (n(s) + 1)^\alpha)$$

where k : expansion rate parameter; α : expansion exponent

Effect: more visits allow more children to be expanded

- **Intelligent action sampling:** (controls “how”)

Strategy: `\_sample\_untried\_action\_uniform(node)`

- Randomly sample candidate actions
- Test action quality using `\_env\_simulate()` to extract collision `info`.
- Prioritize collision-free actions when:
`info['num\_collisions'] + info['num\_vehicle\_collisions'] = 0`
- Fallback to pure random sampling of no-good action found in 50 attempts.

3. Simulation (Rollout):

Method: Smart Rollout with simulation-guided action selection

Strategy: instead of pure random rollout, use heuristic evaluation at each step

Rollout Policy: ``_select_smart_action_via_simulation ()``

- At each state, sample 20 candidate actions
- Evaluate each action's immediate reward (via ``env.simulate``)
- Select and execute the action with the highest composite score

Evaluation criteria:

- Base reward: $R(s,a)$ the immediate reward returned by ``env.simulate``
- Goal achievement bonus: detect state transition from “not reached” to “reached”
- Progress bonus/penalty: moving closer/away from goal (**Chebyshev distance**)
- Movement bonus: any position change to prevent drones from staying stationary
- Collision penalty: penalizes both obstacle collisions and inter-drone collisions
- Continue until terminal or depth limit reached.

Value computation: total expected return during a rollout

The estimated total value obtained after taking action a in state s is given by:

$$v = R(s, a) + \gamma \cdot v_{tail}$$

where v_{tail} : estimated future value from continuing the rollout

4. Backpropagation: Updating the value of the nodes visited in this iteration.

Method: Incremental Average Update

Update formula:

$$Q(s, a) = (Q(s, a) \times N(s, a) + v) / (N(s, a) + 1)$$

$$N(s) = N(s) + 1$$

$$N(s, a) = N(s, a) + 1$$

Pseudo-code:

- Main data structures:

Structure Node:

```
state: State           // State representation
N: Integer             // Visit count
children: Map<Action, EdgeInfo> // Child mapping
is_terminal: Boolean   // Terminal state flag
```

Structure EdgeInfo:

```
N: Integer           // Edge visit count
Q: Float             // Average cumulative return
child: Node          // Child node pointer
```

Global Variables:

```
nodes: Map<StateHash, Node> // State node table
C: Float                   // UCT exploration constant
γ: Float                   // Discount factor
max_depth                 // Maximum search depth
k: Float                  // Progressive widening parameter
α: Float                  // Progressive widening exponent
A: Integer                // Action space size
```

- Algorithm 1: Main planning function

ALGORITHM **Plan**($s_{current}$, time_budget)

INPUT:

```
 $s_{current}$ : Current state
time_budget: Planning time budget (seconds)
```

OUTPUT:

```
a_best: Best action
```

```

BEGIN
    root ← GetOrCreateNode(s_current)      // Get or create root node
    deadline ← CurrentTime() + time_budget // Calculate deadline

    // MCTS main loop: repeated build search tree within time budget
    WHILE CurrentTime() < deadline DO
        _mcts(root, depth=0)
    END WHILE

    // Final action selection
    IF root.children is EMPTY THEN
        RETURN RandomAction()
    END IF

    // Selection strategy: based on visit counts
    a_best ← argmax_{a ∈ root.children} N(s_current, a)

    RETURN a_best
END

```

- Algorithm2: MCTS simulation (core recursive function)

ALGORITHM _mcts(node, depth)

INPUT:

node: Current node

depth: Current search depth

OUTPUT:

v: Estimated value

BEGIN

// ===== Base Cases =====

// Terminal state

IF node.is_terminal THEN

 RETURN 0.0

END IF

// ===== Progressive Widening Check =====

// Calculate maximum allowed children

max_children ← floor($k \cdot (\text{node.N} + 1)^\alpha$)

// ===== EXPANSION Phase =====

IF |node.children| < min(A, max_children) THEN

 // Sample an untried action

 a_new ← **SampleUntriedAction**(node)

 IF a_new ≠ NULL THEN

 // Execute action, get next state (via env.simulate)

 s_next, r, done ← **Forward**(node.state, a_new)

 // Get or create child node

 child ← **GetOrCreateNode**(s_next)

 child.is_terminal ← done

 // Create new edge

 edge ← **EdgeInfo**(N=0, Q=0.0, child=child)

 node.children[a_new] ← edge


```

// ===== SIMULATION (Rollout) =====

IF done THEN
    v_tail ← 0.0
ELSE
    v_tail ← SmartRollout(s_next, depth+1) //rollout value
END IF

// Calculate total value=immediate reward+discounted future value
v ← r + γ · v_tail

// ===== BACKPROPAGATION (Update) =====
edge.Q ← (edge.Q*edge.N+v)/(edge.N+1)
node.N ← node.N + 1
edge.N ← edge.N + 1

RETURN v
END IF
END IF

// ===== SELECTION Phase =====

a_sel, edge_sel ← UCT_Select(node) //uct selection

// Execute selected action (via env.simulate) to get the next state
s_next, r, done ← Forward(node.state, a_sel)
child ← edge_sel.child
child.is_terminal ← done

// Recursively simulate child node (done: reach goal?)
IF done THEN
    v_child ← 0.0
ELSE
    v_child ← mcts(child, depth+1)
END IF

// Calculate total value
v ← r + γ · v_child

// ===== BACKPROPAGATION (Update) =====

// Incremental average update
node.N ← node.N + 1
edge_sel.Q ← (edge_sel.Q · edge_sel.N + v) / edge_sel.N + 1
edge_sel.N ← edge_sel.N + 1

RETURN v
END

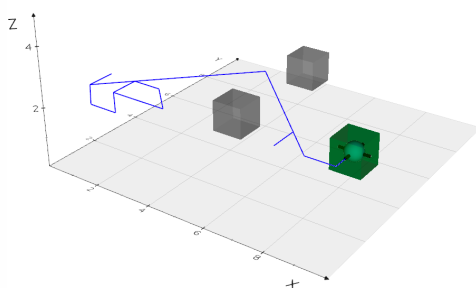
```

3) Refer to the submitted code.

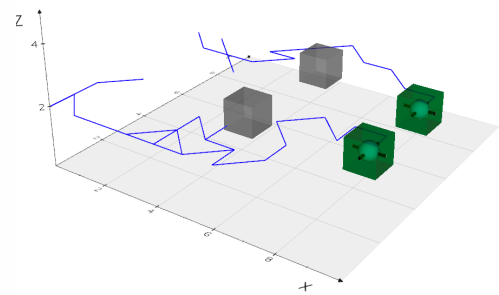
- 4) **The primary objective** is to investigate how increasing the size of the action space (increasing the number of drones) influences the performance of the MDP solver.

Experimental setup:

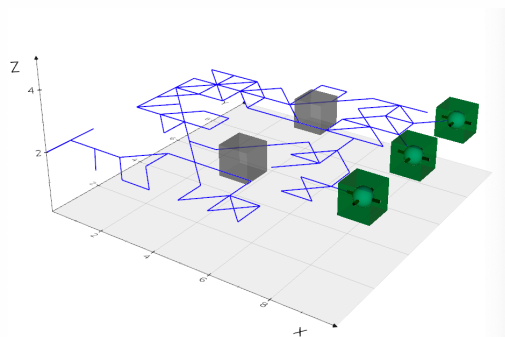
- Trials per setting: 30 runs
- Controlled variables:
 - a) **Fixed** altitude; each drone moves to one of its 8 neighbour XY cells per step.
 - b) Grid size, obstacle layout, environment, and MDP model parameters **held constant**.
 - c) Drone Arrangement: start and goal positions are both evenly spaced along the y-axis, with drones maintaining a straight-line formation at a constant altitude.
- Independent variable:
Number of drones $\in \{1, 2, 3, 4\}$.
(Note: joint action space grows as 8^K , where K is the number of drones.)



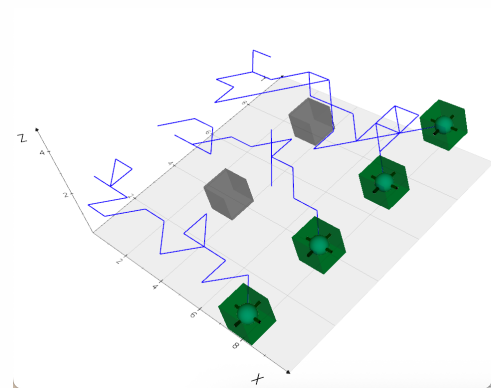
config_1_drones.yaml



config_2_drones.yaml



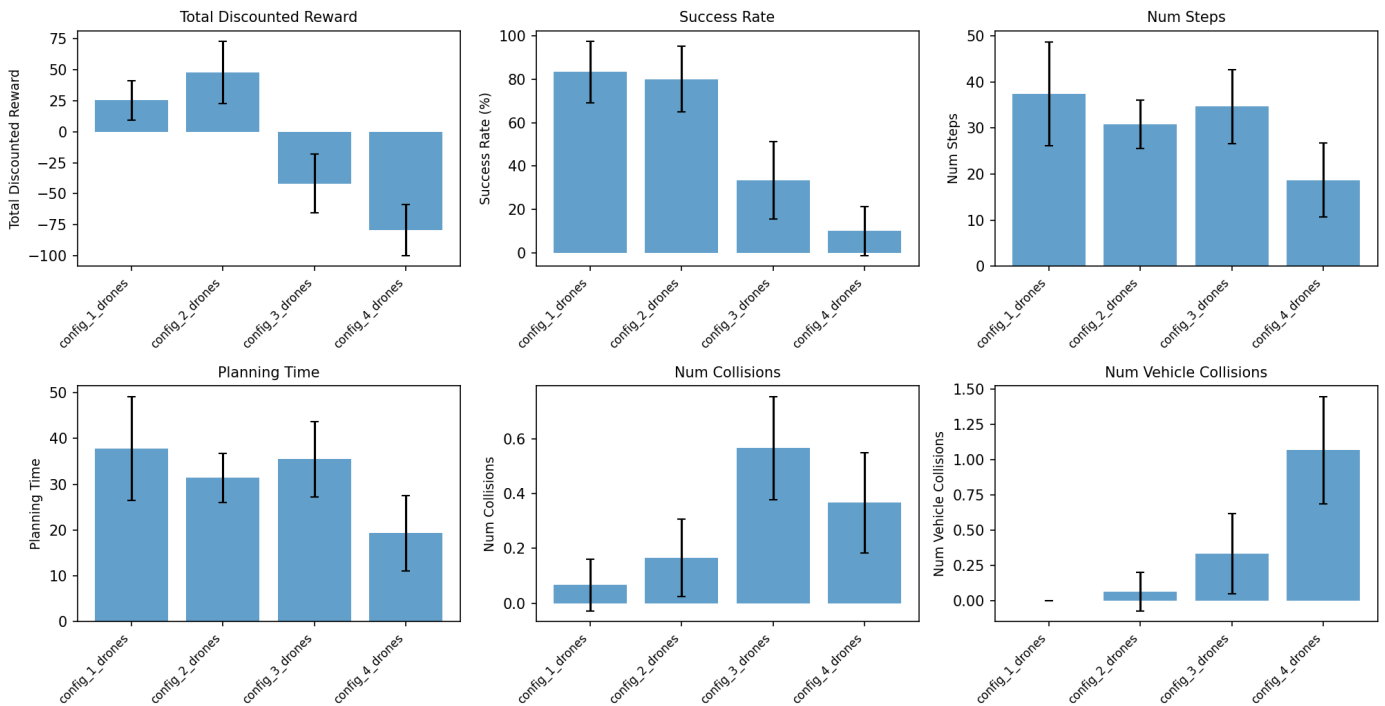
config_3_drones.yaml



config_4_drones.yaml

- Metrics Recorded:
 - a) Success
 - b) Total discounted Reward
 - c) Number of steps
 - d) Number of obstacle collisions
 - e) Number of vehicle collisions
 - f) Planning_time
- Summary Statistics:
All values reported as: Mean $\pm$ half-width (95% Confidence Interval: [lower,upper])

# of drones	Success Rate	Total discounted reward	# of steps	# of obstacle collisions	# of vehicle collisions	Planning time
1 drone	(25/30) 83.3% $\pm$ 14.15% [0.692, 0.975]	25.4200 $\pm$ 15.8288 [9.5912, 41.2488]	37.4667 $\pm$ 11.2345 [26.232, 48.701]	0.0667 $\pm$ 0.0945 [-0.028, 0.161]	0.0000 $\pm$ 0.0000 [0.000, 0.000]	37.7711 $\pm$ 11.2940 [26.477, 49.065]
2 drones	(24/30) 80% $\pm$ 15.20% [0.648, 0.952]	47.6909 $\pm$ 25.1670 [22.5240, 72.8580]	30.8000 $\pm$ 5.2280 [25.572, 36.028]	0.1667 $\pm$ 0.1415 [0.025, 0.308]	0.0667 $\pm$ 0.1365 [-0.070, 0.203]	31.3909 $\pm$ 5.3130 [26.078, 36.704]
3 drones	(10/30) 33.3% $\pm$ 17.90% [0.154, 0.512]	-41.8266 $\pm$ 23.6685 [-65.4950, -18.1580]	34.6667 $\pm$ 8.0370 [26.630, 42.704]	0.5667 $\pm$ 0.1885 [0.378, 0.755]	0.3333 $\pm$ 0.2830 [0.050, 0.616]	35.4674 $\pm$ 8.1765 [27.291, 43.644]
4 drones	(3/30) 10.0% $\pm$ 11.40% [-0.014, 0.214]	-79.2162 $\pm$ 20.6845 [-99.9010, -58.5320]	18.7333 $\pm$ 8.0505 [10.683, 26.784]	0.3667 $\pm$ 0.1830 [0.184, 0.550]	1.0667 $\pm$ 0.3790 [0.688, 1.446]	19.3007 $\pm$ 8.2405 [11.060, 27.541]



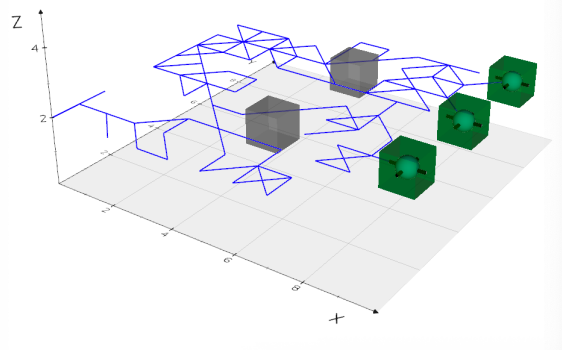
- Discussion: Impact of Action Space Growth on Solver Performance

As the number of drones increases, the size of the joint action space grows exponentially (from 8 actions at $K = 1$ to 4096 at $K = 4$). This rapid expansion severely degrades the efficiency of the MCTS-UCT solver under a fixed per-step planning budget. The algorithm cannot explore the vastly larger set of possible joint actions deeply enough to produce reliable value estimates. Consequently, the search becomes shallow and dominated by noise, leading to erratic root decisions. Empirically, this manifests as a sharp drop in success rate (from 83% at $K = 1$ to only 10% at $K = 4$), an increase in obstacle and inter-drone collisions, and negative total discounted rewards for larger teams. Although total planning time and step count appear to decrease at $K = 4$, this reflects premature termination due to failure rather than improved efficiency. These outcomes highlight the core limitation of UCT-based solvers when the branching factor explodes: progressive widening can only mitigate, not overcome, the combinatorial growth of actions. Overall, the results confirm that the solver scales poorly with action-space size, and its performance deteriorates rapidly as the coordination complexity among agents increases under fixed computational resources.

- 5) **The primary objective** is to investigate how increasing the time per planning step influences the performance of the MDP solver.

Experimental setup:

- Trials per setting: 30 runs
- Controlled variables:
Fixed scenario: 3-drone straight-line formation at a constant altitude, using the same grid, obstacle layout, and MDP parameters as in Question 4.
- Independent variable:
The time per planning step $\in \{0.5, 1.5, 2.5, 3.5\}$.

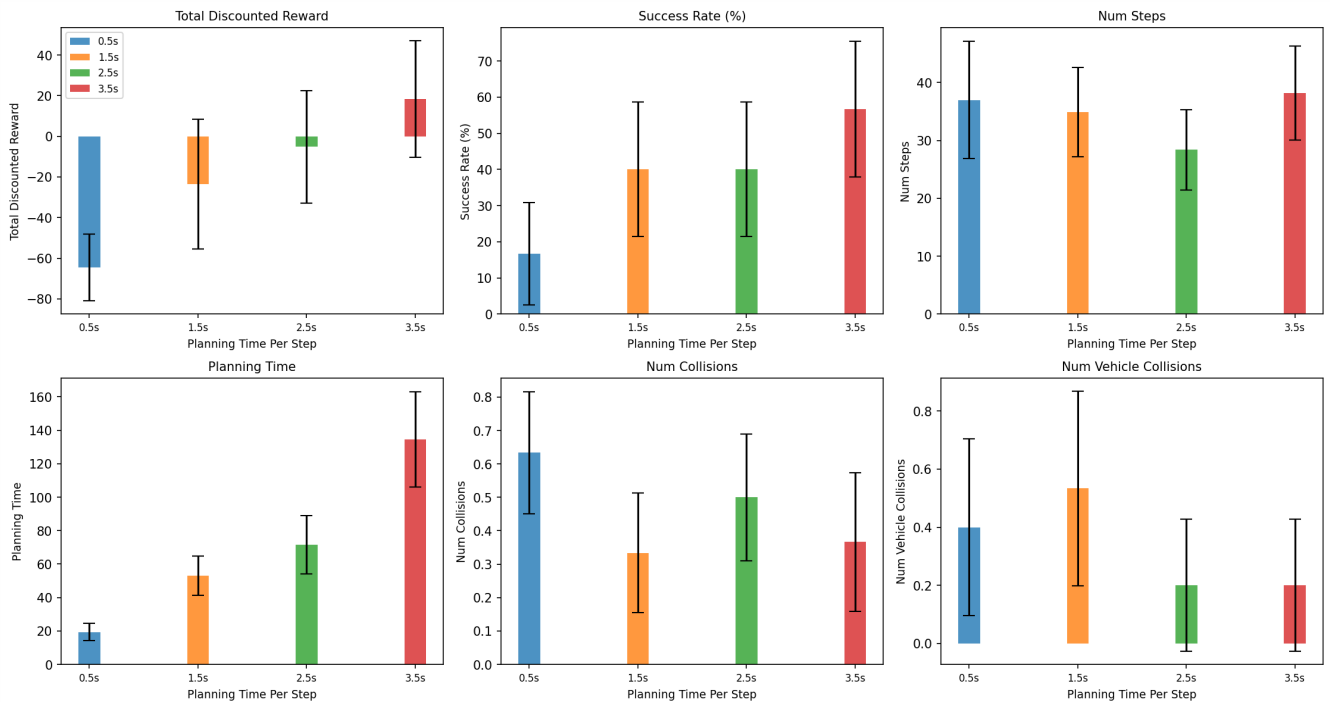


3_drones.yaml

- Metrics Recorded:
 - Success
 - Total discounted Reward
 - Number of steps
 - Number of obstacle collisions
 - Number of vehicle collisions
 - Planning_time
- Summary Statistics:
All values reported as: Mean $\pm$ half-width (95% Confidence Interval: [lower,upper])

Performance vs. Planning Time per Step — config_3_drones

Planning Time (s)	Success Rate	Total Discounted Reward	Steps	Obstacle Collisions	Vehicle Collisions	Total Planning Time
0.5	(5/30) 16.7% $\pm$ 14.2 [2.5%, 30.8%]	-64.53 $\pm$ 16.48 [-81.01, -48.05]	37.00 $\pm$ 10.14 [26.86, 47.14]	0.63 $\pm$ 0.18 [0.45, 0.82]	0.40 $\pm$ 0.30 [0.10, 0.70]	19.38 $\pm$ 5.26 [14.12, 24.64]
1.5	(12/30) 40.0% $\pm$ 18.6 [21.4%, 58.6%]	-23.45 $\pm$ 31.92 [-55.37, 8.48]	34.87 $\pm$ 7.69 [27.18, 42.56]	0.33 $\pm$ 0.18 [0.15, 0.51]	0.53 $\pm$ 0.34 [0.20, 0.87]	53.14 $\pm$ 11.68 [41.45, 64.82]
2.5	(12/30) 40.0% $\pm$ 18.6 [21.4%, 58.6%]	-5.17 $\pm$ 27.67 [-32.84, 22.50]	28.37 $\pm$ 6.97 [21.39, 35.34]	0.50 $\pm$ 0.19 [0.31, 0.69]	0.20 $\pm$ 0.23 [-0.03, 0.43]	71.59 $\pm$ 17.57 [54.02, 89.15]
3.5	(17/30) 56.7% $\pm$ 18.9 [37.8%, 75.5%]	18.35 $\pm$ 28.71 [-10.36, 47.06]	38.20 $\pm$ 8.11 [30.09, 46.31]	0.37 $\pm$ 0.21 [0.16, 0.57]	0.20 $\pm$ 0.23 [-0.03, 0.43]	134.56 $\pm$ 28.54 [106.02, 163.10]



- Discussion: Effect of Increasing Planning Time per Step on Solver Performance

Increasing the per-step planning time significantly enhances the performance of the MCTS-UCT solver by allowing more extensive exploration and deeper value propagation within the search tree. As the time budget grows from 0.5 s to 3.5 s, the solver can conduct more rollouts, expand additional child nodes through progressive widening, and perform more accurate backpropagation of value estimates. This leads to a clear rise in success rate (from 16.7% to 56.7%) and a shift in total discounted reward from strongly negative to positive, indicating improved decision quality and goal attainment. The reduction in both obstacle and inter-drone collisions further suggests that longer computation time improves coordination and foresight in joint-action selection. However, the benefits exhibit diminishing returns beyond roughly 2–3s per step, where the confidence intervals overlap and total planning time grows disproportionately. This plateau reflects a computational saturation point where additional rollouts contribute little new information relative to cost. Overall, the results highlight a fundamental trade-off: longer planning steps yield higher accuracy and stability but at rapidly increasing computational expense, and the solver achieves near-optimal efficiency around the 2–3s “knee point” in the time–performance curve.

6) The **limitation** of the proposed solver:

The proposed MCTS-UCT solver, while effective for small teams and moderate planning budgets, faces several key limitations. First, it scales poorly with the exponential growth of the joint action space as the number of drones increases; progressive widening alleviates but cannot eliminate this combinatorial explosion, leading to shallow exploration and noisy value estimates. Second, under a fixed time budget, the solver’s performance is highly sensitive to stochastic rollouts and random initialization, producing unstable outcomes across trials. Third, its purely simulation-based planning lacks learned priors or heuristic guidance, resulting in inefficient search and redundant exploration. Finally, the solver assumes perfect state observability and ignores real-time communication or dynamics uncertainties, limiting its applicability to complex, partially observable, or continuous domains. Overall, these weaknesses highlight the need for hybrid or learning-guided MCTS approaches to achieve better scalability, stability, and real-world robustness.

Reference:

- The use of AI tool: Gemini: <https://gemini.google.com/share/580e29500e46>